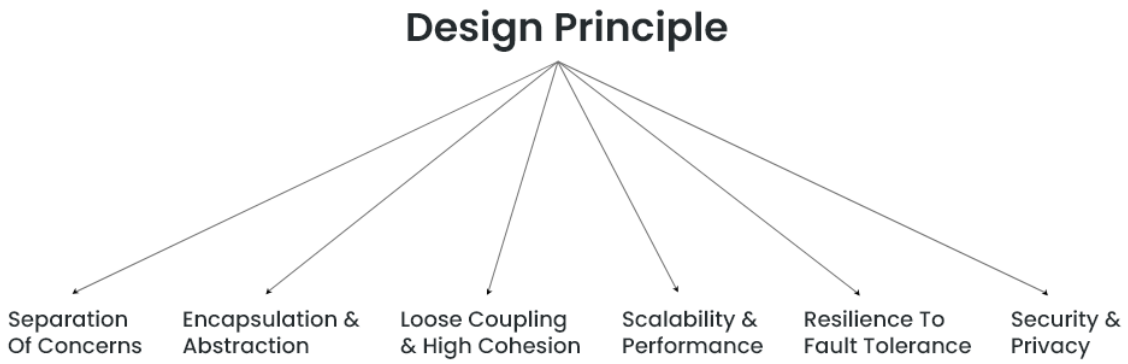
Design Principles in System Design

Design Principles in System Design are a set of considerations that form the basis of any good system. But why use Design Principles in System Design? Design Principles help teams with decision-making. The multi-disciplinary field involves trade-off analysis, balancing conflicting needs, and making decisions about design choices that will impact the overall system.

What are Design Principles in System Design?

System design principles are essential rules used in the development of reliable, expandable, and maintained systems. These guidelines are necessary for making accurate decisions and guaranteeing that the system satisfies the requirements and intended goals.



Design Principles in System Design

Key Design Principles in System Design

Some of the most common Design Principles in System Design are:

1. Separation of Concerns
2. Encapsulation and Abstraction
3. Loose Coupling and High Cohesion
4. Scalability and Performance
5. Resilience to Fault Tolerance
6. Security and Privacy

To gain a better understanding, let's go over each design idea as follows:

Separation of Concerns

Fundamental design principles that encourage code **organization and maintainabilit**y include modularity and separation of concerns. Developers can concentrate on particular parts of a system independently by breaking it up into smaller, self-contained modules, making the system simpler to undersatnd, test, and manage. Each module must have a clearly defined role that encompasses proper functionality and reduces reliance on other modules. This makes it possible to scale or replace particular components without having an adverse effect on the system as a whole, and it also makes maintenance easier.

Benefits and Examples

- **Benefits :** It simplifies the system and make it easier to understand, test and deploy. It enhances the maintainability and scalability.
- **Examples :** Modular programming, where each module handles the specific functionality and reducing the interdependencies.

Encapsulation and Abstraction

Encapsulation and **Abstraction** are fundamental design principles that promote information hiding and reduce system complexity.

- **Encapsulation:** It refers to bundling data (attributes) and related behaviors (methods) into a single unit, such as a **class or module**, while restricting direct access to internal details. This is achieved through access modifiers (e.g., private, protected) and well-defined interfaces, ensuring controlled interaction with the object's state.
- **Abstraction:** It involves simplifying complex systems by modeling only the essential characteristics while hiding unnecessary implementation details. It provides a high-level logical representation, allowing users to interact with systems without understanding their internal complexities.

Loose Coupling and High Cohesion

- **Coupling** refers to the degree of interdependence between software modules. High coupling means that modules are closely connected and changes in one module may affect other modules. Low coupling means that modules are independent and changes in one module have little impact on other modules. It is of two types:
    - Loose coupling
    - Tight coupling
- **Cohesion** refers to the degree to which elements within a module work together to fulfill a single, well-defined purpose. High cohesion means that elements are closely related and focused on a single purpose, while low cohesion means that elements are loosely related and serve multiple purposes.
    - Low cohesion
    - High cohesion

Here while designing principles it is to make sure that *coupling should be loose and cohesion should be high* while designing flexible systems. With loose coupling, we are reducing dependencies between components by minimizing direct communication between them and especially relying on interfaces. Now with high cohesion, the functionality is confined within a module which ensures that components work together for a common goal ensuring reusability and understandability.

Note : For well working system coupling should be loose and Cohesion should be high. And for more details chekout this article.

Scalability and Performance

Building systems that are intended to manage increasing workloads or vast amounts of data requires careful consideration of **scalability and performance**. System designers should take into account both horizontal scaling (adding more instances or nodes) and vertical scaling (raising the resources of a single node) to accomplish scalability. The workload can also be distributed and system responsiveness increased by using strategies like load balancing, caching, and **asynchronous processing**. To achieve optimum performance, it's critical to spot possible bottlenecks early in the design phase and execute the necessary optimizations.

Check this for more info. - Performance vs Scalability in System Design

Resilience and Fault Tolerance

In order to ensure system availability and reliability, it is crucial to design fault tolerance and resilience. Techniques like redundancy, replication, and **fault detection algorithms** are used in this. System downtime can be avoided or minimized by designing systems that can survive component failures and gracefully handle exceptions, hence minimizing the impact of failures. System resilience is further improved by putting backup and recovery procedures in place and conducting careful testing and monitoring.

Read this article for more understnading : Fault tolerance in system design

Privacy and Security

In today's interconnected world, **privacy and security** are critical concerns in system design. Designers must integrate robust security measures at every stage of development to protect sensitive data and prevent unauthorized access. Key strategies include:

- **Encryption**: Securing data in transit and at rest using cryptographic techniques (e.g., AES, RSA, TLS).
- **Authentication & Authorization**: Implementing mechanisms like OAuth, JWT, or multi-factor authentication (MFA) to verify user identities and enforce role-based access control (RBAC).
- **Data Minimization & Anonymization**: Collecting only necessary data and anonymizing personally identifiable information (PII) to reduce privacy risks.
- **Secure APIs & Input Validation**: Preventing injection attacks (e.g., SQLi, XSS) through sanitization and parameterized queries.
- **Audit Logging & Monitoring**: Tracking access patterns and anomalies to detect and respond to breaches proactively.

Applying Design Principles to a Transport System

Introduction to the Transport System Example

A transport system is a complex system that can benefit significantly from the application of design principles. Let's explore how these principles can enhance the user experience and system efficiency.

- **Simplicity:** The transport system should have a clear and easy design that is easy to understand. Clear and simple user-friendly leads to a simplified user experience. The best example of it is Google whose no ads are run over the search main page in order to keep it simple and minimalistic so a user-friendly interface is maintained.

- **Balance:** The system should aim for an equitable distribution of transport options across the city, taking into account elements such as demand, accessibility, and population density. This guarantees that various places are suitably supplied and avoids congestion or underuse of particular routes or modes.
- **Contrast:** To distinguish between different forms of transportation (such as buses, trains, and trams) or routes, contrast can be provided by visual cues and color coding. Users may rapidly find pertinent information and make decisions regarding their journey thanks to this.
- **Unification:** A transportation system should work to achieve unification by maintaining a unified visual identity across all touchpoints, such as cars, signage, and ticketing materials. Users can recognize and trust the system's services more easily because of a consistent design that strengthens brand identification.
- **Functionality:** To maintain seamless operations and user pleasure, the system should put a high priority on functionality. This includes elements like dependable scheduling, seamless transitions between modes of transportation, infrastructure that is accessible to those with disabilities, and thoughtfully constructed platforms and waiting rooms.
- **Typography:** Signage, route maps, and other informational products within the transport system can all benefit from the application of typographic principles to improve readability and effectively transmit information. Clear communication with consumers is facilitated by legal typefaces, suitable font sizes, and acceptable spacing.

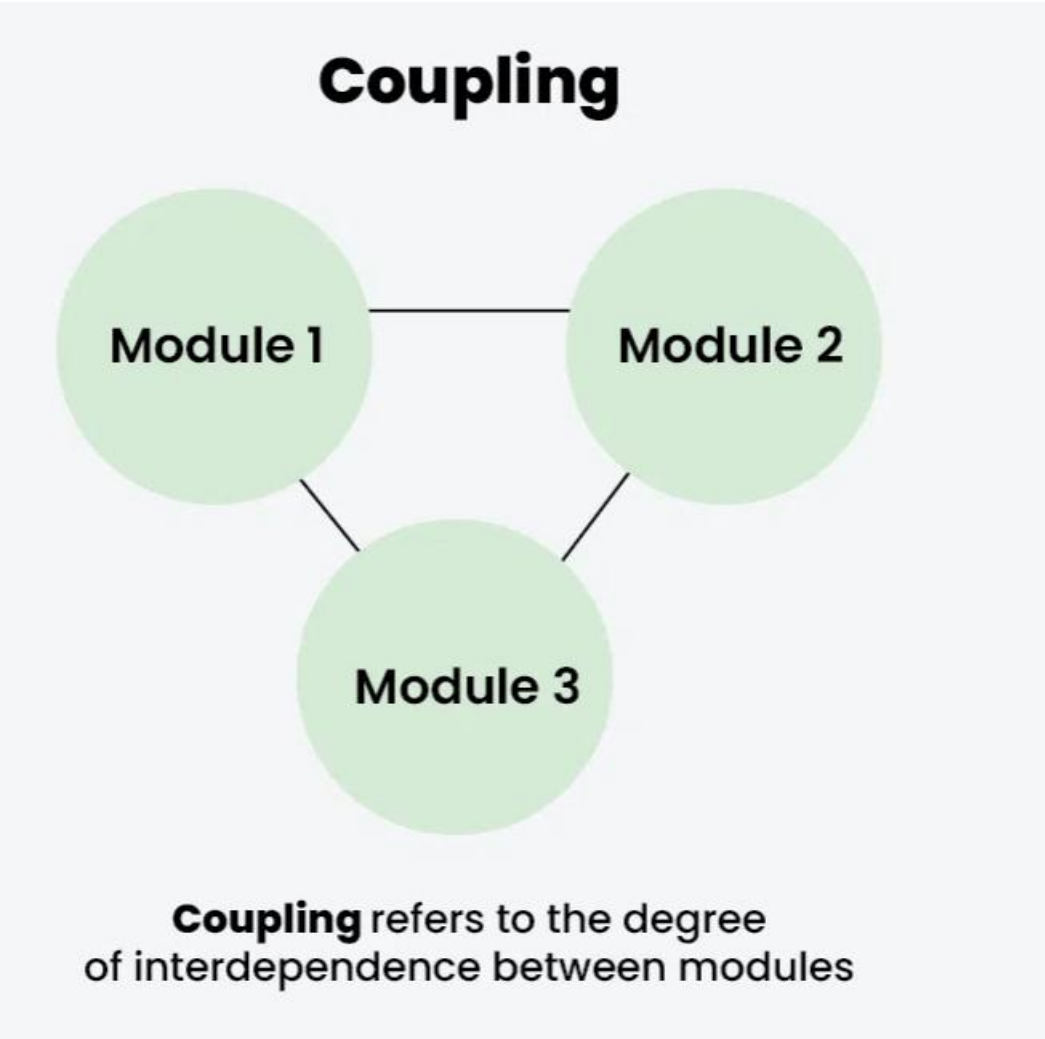
Coupling and Cohesion - Software Engineering

The purpose of the Design phase in the Software Development Life Cycle is to produce a solution to a problem given in the SRS(Software Requirement Specification) document. The output of the design phase is a Software Design Document (SDD).

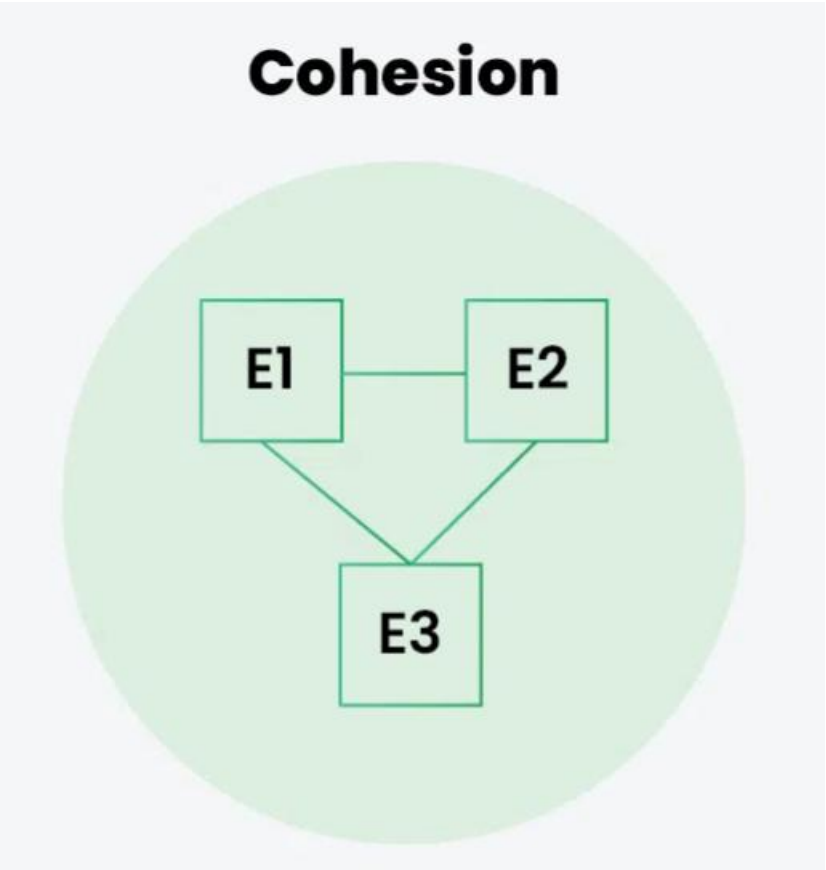
Coupling and Cohesion are two key concepts in software engineering that are used to measure the quality of a software system's design.

What is Coupling and Cohesion?

Coupling refers to the degree of interdependence between software modules. High coupling means that modules are closely connected and changes in one module may affect other modules. Low coupling means that modules are independent, and changes in one module have little impact on other modules.



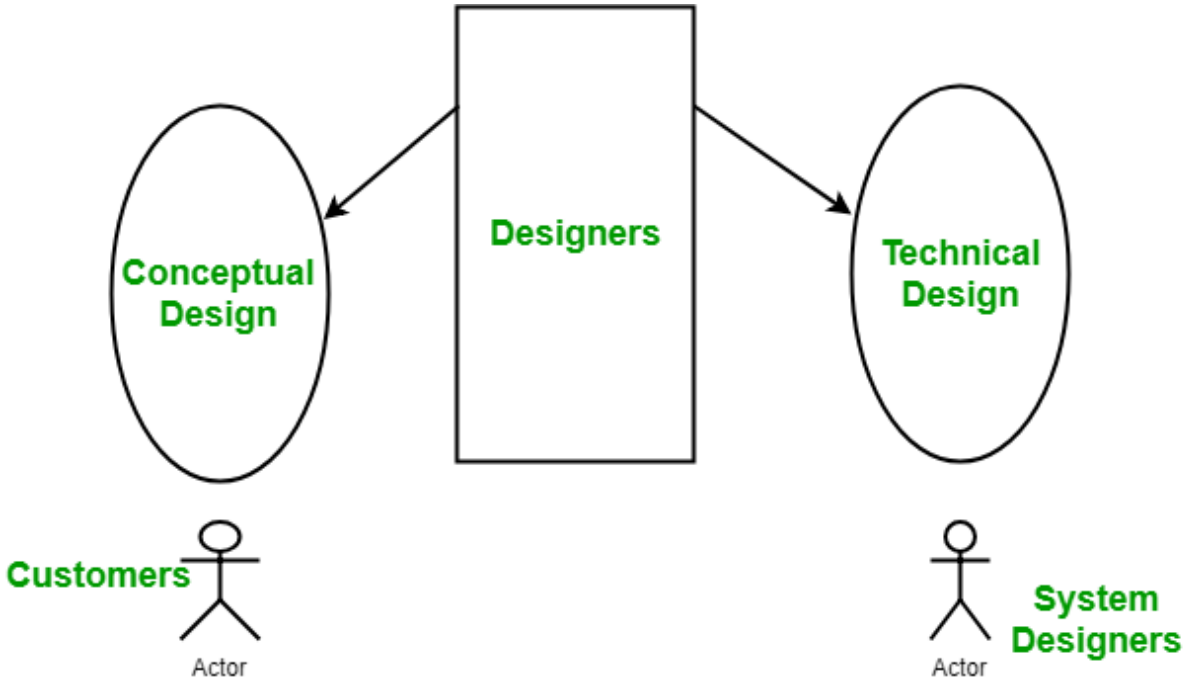
Cohesion refers to the degree to which elements within a module work together to fulfill a single, well-defined purpose. High cohesion means that elements are closely related and focused on a single purpose, while low cohesion means that elements are loosely related and serve multiple purposes.



Cohesion

Both coupling and cohesion are important factors in determining the maintainability, scalability, and reliability of a software system. High coupling and low cohesion can make a system difficult to change and test, while low coupling and high cohesion make a system easier to maintain and improve.

Basically, design is a two-part iterative process. The first part is Conceptual Design which tells the customer what the system will do. Second is Technical Design which allows the system builders to understand the actual hardware and software needed to solve a customer's problem.



Conceptual design of the system:

- Written in simple language i.e. customer understandable language.
- Detailed explanation about system characteristics.
- Describes the functionality of the system.
- It is independent of implementation.
- Linked with requirement document.

Technical Design of the System:

- Hardware component and design.
- Functionality and hierarchy of software components.
- Software architecture
- Network architecture
- Data structure and flow of data.
- I/O component of the system.
- Shows interface.

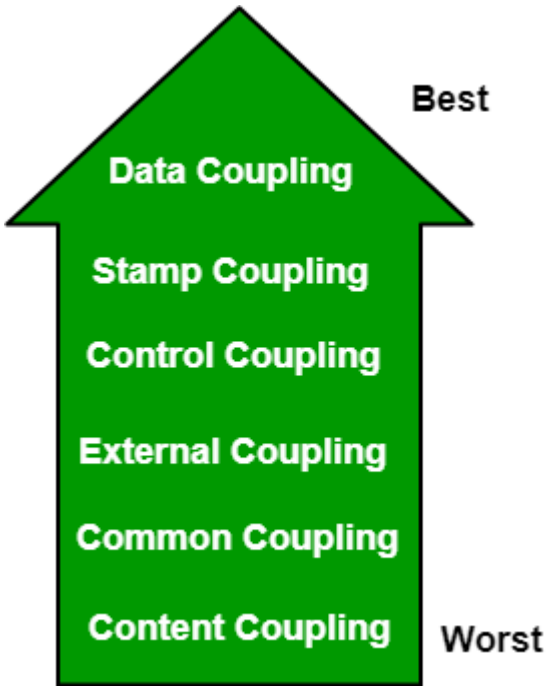
Modularization is the process of dividing a software system into multiple independent modules where each module works independently. There are many advantages of Modularization in software engineering. Some of these are given below:

- Easy to understand the system.
- System maintenance is easy.

- A module can be used many times as their requirements. No need to write it again and again.

Types of Coupling

Coupling is the measure of the degree of interdependence between the modules. A good software will have low coupling.



Types of Coupling

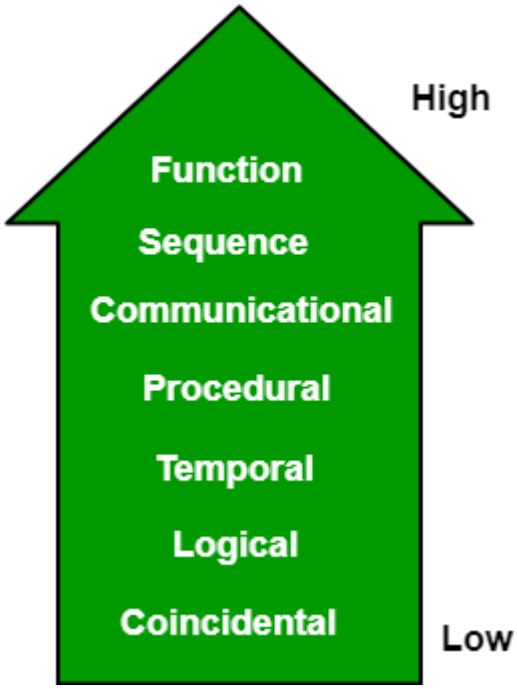
Following are the types of Coupling:

- **Data Coupling:** If the dependency between the modules is based on the fact that they communicate by passing only data, then the modules are said to be data coupled. In data coupling, the components are independent of each other and communicate through data. Module communications don't contain tramp data. Example-customer billing system.
- **Stamp Coupling** In stamp coupling, the complete data structure is passed from one module to another module. Therefore, it involves tramp data. It may be necessary due to efficiency factors- this choice was made by the insightful designer, not a lazy programmer.
- **Control Coupling:** If the modules communicate by passing control information, then they are said to be control coupled. It can be bad if parameters indicate completely different behavior and good if parameters allow factoring and reuse of functionality. Example- sort function that takes comparison function as an argument.
- **External Coupling:** In external coupling, the modules depend on other modules, external to the software being developed or to a particular type of hardware. Ex- protocol, external file, device format, etc.
- **Common Coupling:** The modules have shared data such as global data structures. The changes in global data mean tracing back to all modules which access that data to evaluate the effect of the change. So it has got disadvantages like difficulty in reusing modules, reduced ability to control data accesses, and reduced maintainability.

- **Content Coupling:** In a content coupling, one module can modify the data of another module, or control flow is passed from one module to the other module. This is the worst form of coupling and should be avoided.
- **Temporal Coupling:** Temporal coupling occurs when two modules depend on the timing or order of events, such as one module needing to execute before another. This type of coupling can result in design issues and difficulties in testing and maintenance.
- **Sequential Coupling:** Sequential coupling occurs when the output of one module is used as the input of another module, creating a chain or sequence of dependencies. This type of coupling can be difficult to maintain and modify.
- **Communicational Coupling:** Communicational coupling occurs when two or more modules share a common communication mechanism, such as a shared message queue or database. This type of coupling can lead to performance issues and difficulty in debugging.
- **Functional Coupling:** Functional coupling occurs when two modules depend on each other's functionality, such as one module calling a function from another module. This type of coupling can result in tightly-coupled code that is difficult to modify and maintain.
- **Data-Structured Coupling:** Data-structured coupling occurs when two or more modules share a common data structure, such as a database table or data file. This type of coupling can lead to difficulty in maintaining the integrity of the data structure and can result in performance issues.
- **Interaction Coupling:** Interaction coupling occurs due to the methods of a class invoking methods of other classes. Like with functions, the worst form of coupling here is if methods directly access internal parts of other methods. Coupling is lowest if methods communicate directly through parameters.
- **Component Coupling:** Component coupling refers to the interaction between two classes where a class has variables of the other class. Three clear situations exist as to how this can happen. A class C can be component coupled with another class C1, if C has an instance variable of type C1, or C has a method whose parameter is of type C1,or if C has a method which has a local variable of type C1. It should be clear that whenever there is component coupling, there is likely to be interaction coupling.

Types of Cohesion

Cohesion is a measure of the degree to which the elements of the module are functionally related. It is the degree to which all elements directed towards performing a single task are contained in the component. Basically, cohesion is the internal glue that keeps the module together. A good software design will have high cohesion.



Types of Cohesion

Following are the Types of Cohesion:

- **Functional Cohesion:** Every essential element for a single computation is contained in the component. A functional cohesion performs the task and functions. It is an ideal situation.
- **Sequential Cohesion:** An element outputs some data that becomes the input for other element, i.e., data flow between the parts. It occurs naturally in functional programming languages.
- **Communicational Cohesion:** Two elements operate on the same input data or contribute towards the same output data. Example- update record in the database and send it to the printer.
- **Procedural Cohesion:** Elements of procedural cohesion ensure the order of execution. Actions are still weakly connected and unlikely to be reusable. Ex- calculate student GPA, print student record, calculate cumulative GPA, print cumulative GPA.
- **Temporal Cohesion:** The elements are related by their timing involved. A module connected with temporal cohesion all the tasks must be executed in the same time span. This cohesion contains the code for initializing all the parts of the system. Lots of different activities occur, all at unit time.
- **Logical Cohesion:** The elements are logically related and not functionally. Ex- A component reads inputs from tape, disk, and network. All the code for these functions is in the same component. Operations are related, but the functions are significantly different.
- **Coincidental Cohesion:** The elements are not related(unrelated). The elements have no conceptual relationship other than location in source code. It is accidental and the worst form of cohesion. Ex- print next line and reverse the characters of a string in a single component.

- **Procedural Cohesion:** This type of cohesion occurs when elements or tasks are grouped together in a module based on their sequence of execution, such as a module that performs a set of related procedures in a specific order. Procedural cohesion can be found in structured programming languages.
- **Communicational Cohesion:** Communicational cohesion occurs when elements or tasks are grouped together in a module based on their interactions with each other, such as a module that handles all interactions with a specific external system or module. This type of cohesion can be found in object-oriented programming languages.
- **Temporal Cohesion:** Temporal cohesion occurs when elements or tasks are grouped together in a module based on their timing or frequency of execution, such as a module that handles all periodic or scheduled tasks in a system. Temporal cohesion is commonly used in real-time and embedded systems.
- **Informational Cohesion:** Informational cohesion occurs when elements or tasks are grouped together in a module based on their relationship to a specific data structure or object, such as a module that operates on a specific data type or object. Informational cohesion is commonly used in object-oriented programming.
- **Functional Cohesion:** This type of cohesion occurs when all elements or tasks in a module contribute to a single well-defined function or purpose, and there is little or no coupling between the elements. Functional cohesion is considered the most desirable type of cohesion as it leads to more maintainable and reusable code.
- **Layer Cohesion:** Layer cohesion occurs when elements or tasks in a module are grouped together based on their level of abstraction or responsibility, such as a module that handles only low-level hardware interactions or a module that handles only high-level business logic. Layer cohesion is commonly used in large-scale software systems to organize code into manageable layers.

Advantages of Low coupling

- Improved maintainability: Low coupling reduces the impact of changes in one module on other modules, making it easier to modify or replace individual components without affecting the entire system.
- Enhanced modularity: Low coupling allows modules to be developed and tested in isolation, improving the modularity and reusability of code.
- Better scalability: Low coupling facilitates the addition of new modules and the removal of existing ones, making it easier to scale the system as needed.

Advantages of High cohesion

- Improved readability and understandability: High cohesion results in clear, focused modules with a single, well-defined purpose, making it easier for developers to understand the code and make changes.
- Better error isolation: High cohesion reduces the likelihood that a change in one part of a module will affect other parts, making it easier to
- Improved reliability: High cohesion leads to modules that are less prone to errors and that function more consistently,

- leading to an overall improvement in the reliability of the system.

Disadvantages of High coupling

- Increased complexity: High coupling increases the interdependence between modules, making the system more complex and difficult to understand.
- Reduced flexibility: High coupling makes it more difficult to modify or replace individual components without affecting the entire system.
- Decreased modularity: High coupling makes it more difficult to develop and test modules in isolation, reducing the modularity and reusability of code.

Disadvantages of Low cohesion

- Increased code duplication: Low cohesion can lead to the duplication of code, as elements that belong together are split into separate modules.
- Reduced functionality: Low cohesion can result in modules that lack a clear purpose and contain elements that don't belong together, reducing their functionality and making them harder to maintain.
- Difficulty in understanding the module: Low cohesion can make it harder for developers to understand the purpose and behavior of a module, leading to errors and a lack of clarity.

Architectural Design - Software Engineering

The software needs an architectural design to represent the design of the software. IEEE defines architectural design as "the process of defining a collection of hardware and software components and their interfaces to establish the framework for the development of a computer system." The software that is built for computer-based systems can exhibit one of these many architectural styles.

<u>System Category Consists of</u>

- A set of components(eg: a database, computational modules) that will perform a function required by the system.
- The set of connectors will help in coordination, communication, and cooperation between the components.
- Conditions that defines how components can be integrated to form the system.
- Semantic models that help the designer to understand the overall properties of the system.

The use of architectural styles is to establish a structure for all the components of the system.

<u>Taxonomy of Architectural Styles</u>

1] Data centered architectures:

- A data store will reside at the center of this architecture and is accessed frequently by the other components that update, add, delete, or modify the data present within the store.
- The figure illustrates a typical data-centered style. The client software accesses a central repository. Variations of this approach are used to transform the repository into a blackboard

when data related to the client or data of interest for the client change the notifications to client software.

- This data-centered architecture will promote integrability. This means that the existing components can be changed and new client components can be added to the architecture without the permission or concern of other clients.
- Data can be passed among clients using the blackboard mechanism.

Advantages of Data centered architecture:

- Repository of data is independent of clients
- Client work independent of each other
- It may be simple to add additional clients.
- Modification can be very easy

Data centered architecture

2] Data flow architectures:

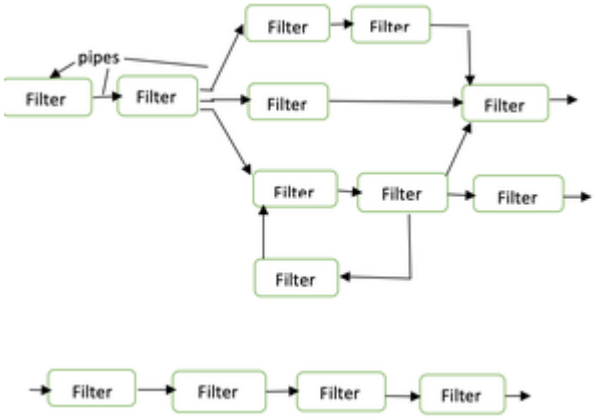
- This kind of architecture is used when input data is transformed into output data through a series of computational manipulative components.
- The figure represents pipe-and-filter architecture since it uses both pipe and filter and it has a set of components called filters connected by lines.
- Pipes are used to transmitting data from one component to the next.
- Each filter will work independently and is designed to take data input of a certain form and produces data output to the next filter of a specified form. The filters don't require any knowledge of the working of neighboring filters.
- If the data flow degenerates into a single line of transforms, then it is termed as batch sequential. This structure accepts the batch of data and then applies a series of sequential components to transform it.

Advantages of Data Flow architecture:

- It encourages upkeep, repurposing, and modification.
- With this design, concurrent execution is supported.

Disadvantage of Data Flow architecture:

- It frequently degenerates to batch sequential system
- Data flow architecture does not allow applications that require greater user engagement.
- It is not easy to coordinate two different but related streams

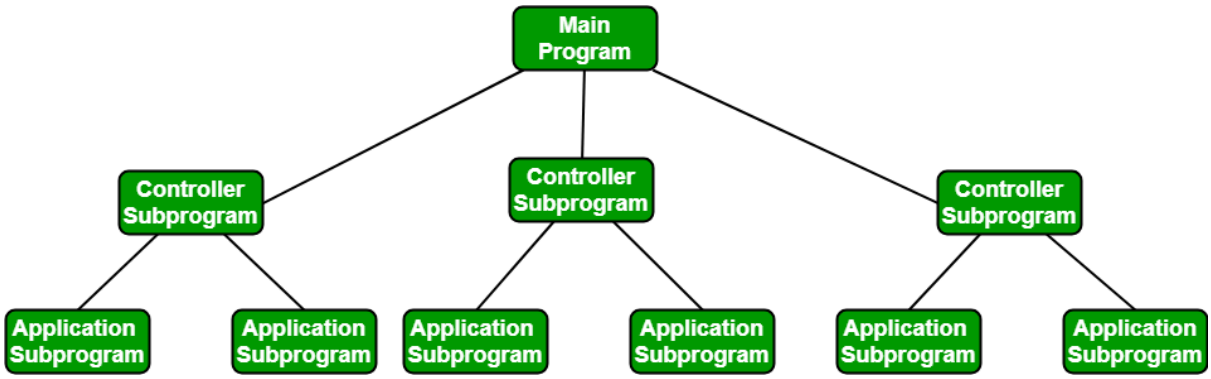


Data Flow architecture

3] Call and Return architectures

It is used to create a program that is easy to scale and modify. Many sub-styles exist within this category. Two of them are explained below.

- **Remote procedure call architecture:** This components is used to present in a main program or sub program architecture distributed among multiple computers on a network.
- **Main program or Subprogram architectures:** The main program structure decomposes into number of subprograms or function into a control hierarchy. Main program contains number of subprograms that can invoke other components.



4] Object Oriented architecture

The components of a system encapsulate data and the operations that must be applied to manipulate the data. The coordination and communication between the components are established via the message passing.

Characteristics of Object Oriented architecture:

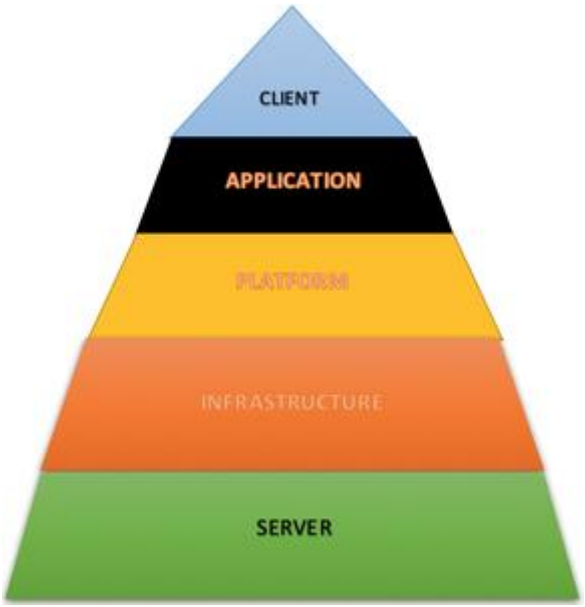
- Object protect the system's integrity.
- An object is unaware of the depiction of other items.

Advantage of Object Oriented architecture:

- It enables the designer to separate a challenge into a collection of autonomous objects.
- Other objects are aware of the implementation details of the object, allowing changes to be made without having an impact on other objects.

5] Layered architecture

- A number of different layers are defined with each layer performing a well-defined set of operations. Each layer will do some operations that becomes closer to machine instruction set progressively.
- At the outer layer, components will receive the user interface operations and at the inner layers, components will perform the operating system interfacing(communication and coordination with OS)
- Intermediate layers to utility services and application software functions.
- One common example of this architectural style is OSI-ISO (Open Systems Interconnection-International Organisation for Standardisation) communication system.



Layered architecture

Transaction Mapping - Software Engineering

Transaction mapping has turned out to be a decisive aspect in the Software Engineering arena. It guarantees that the business processes are properly matched, the design does not use too many resources, the data is accurate, the communication between the departments is effective, and there's room for scaling the business up and testing/validation processes that maintain quality. Here we assume we talk about a development process consisting of a structured chain of steps and let's imagine that an example of such a process is shopping from the online store. This way, the blocks would be more accurate in transactions, becoming more stable and complete.

What is Transaction Mapping in Software Engineering?

Mapping transactions to software in an engineering context implies establishing relationships between business transactions and precise software parts and functioning features. This is the most important aspect of Software system developability, and it is a core element of business process automation.

1. **Understanding Business Transactions:** In programming, the business transaction is anything that the company does which resembles an agreement needed to have an outcome needed by the business process. For instance, ordering or making a payment is a type of process in the day-to-day operations, as well as producing a report.

2. **Mapping to Software Components:** Transaction mapping, on the other hand, implies that identify the software modules, functions and transaction lines where transactions are generators. It involves documenting all the steps, or business process activities, executed to make sure they effectively work within the software system environment.

3. **Data Flow and Processing Logic:** Transaction mapping concerns with how information is transmitted as well as the logic of what to process for each business transaction. This involves data communication being safeguarded through data collection, processing, access and distribution wiring to result in calculations and systems operating effectively and efficiently.

4. **Ensuring Accuracy and Efficiency:** This stage of transaction mapping is meant to verify whether the transactions are complete as well as accurate with respect to the processing steps that are linked with the software which is supporting the actual business operations. It should comprehend the specifics of each transaction system and what type of information moves (inputs, outputs, processing, and handling of errors) and provide for the security aspect as well.

Why Transaction Mapping is Important?

Transaction mapping is essential in Software Development for various reasons:

1. **Alignment with Business Processes:** Firstly, it validates the software systems by making sure that they accurately capture the actual business steps. This shows the architecture is imperative for the office to operate effective, have no waste of resources and in accordance with its user needs.

2. **Efficient Software Design:** Software modules are more perfectly made by making their building on business operations, the developer can do this in a logical and efficient way. Consequently, we cut off code complexity, ease in both maintenance and further enhancements.

3. **Accuracy and Data Integrity:** Data modeling must be set up in a way that ensures standardized data input and correct performance of calculations. The end result is data integrity and accuracy. Such a feature is crucial in this scenario helping the analysts understand the data and keep it aligned with the practice.

4. **Effective Communication:** Earlier communication between stakeholders such as business analysts, developers and customer also is enabled via Transactions mapping. It is more than just a process of creating the code; it also defines the requirements, dependencies, and shared understanding of how the software will be able to work.

5. **Scalability and Flexibility:** A thoughtfully made transactions mapping implements systems of adaptability in the future and allows for software systems to adjust to new requirements. This enables one to achieve his objectives much faster, roll out new features, introduce innovational processes and address the changing business needs.
6. I**mproved Testing and Validation:** Transactional testing is in place to find out if the whole system has been thoroughly tested, including boundary cases and validations. This leads to a propter and stable system all together.

Steps of Transaction Mapping

Step 1: Review the fundamental system model.

To begin with, present the base system model allowing the participants to observe and understand the architecture, sequence, and interactions of its components Thus planing stage of the mapping infrastructure creation gives a short look on the system's functions and, in a long run, helps to obtain data of transaction mapping.

Step 2: Review and refine data flow diagrams (DFDs) for the software.

DFDs(Data flow Diagrams) which depict the primary processes and interactions of system elements should be studied. Create these diagrams up-to-date so as to show exactly the system logic functioning.

Step 3: Determine whether the DFD has transform or transaction flow characteristics.

Note whether the DFD is representing the transform flows (data processing) or the transaction flows (action triggered by a marker). This becoming possible a developer can track every link between two given data points in the system.

Step 4: Identify the transaction center and the flow characteristics along each action path.

Clearly designate the nodes (action paths) which have transaction tables first and further categorize them as short-term or long-term. Know which data elements trigger data circulation and feed workflows.

Step 5: Map the DFD in a program structure amenable to transaction processing.

Create a programming design from DFD that will be able to process and manage transactions. Defining such building blocks like data validation and storage methods, become important in the process of creating a virtual world.

Step 6: Factor and refine the transaction structure and the structure of each action path.

Streamline the process of authorities in transactions using tree (Patricia tree) for data manipulation, management of consistency and performance. Modify the channel structure of the operational route for each business data processing to guarantee the integrity of the route and eliminate the bottlenecks.

Step 7: Refine the first-iteration architecture using design heuristics for improved software quality.

Improve the initial architecture by incorporating conceptual heuristics, best software practices, quality barometers, etc. Look into issues with modular bits that can be employed again, security and scalability to boost up the general quality of the application.

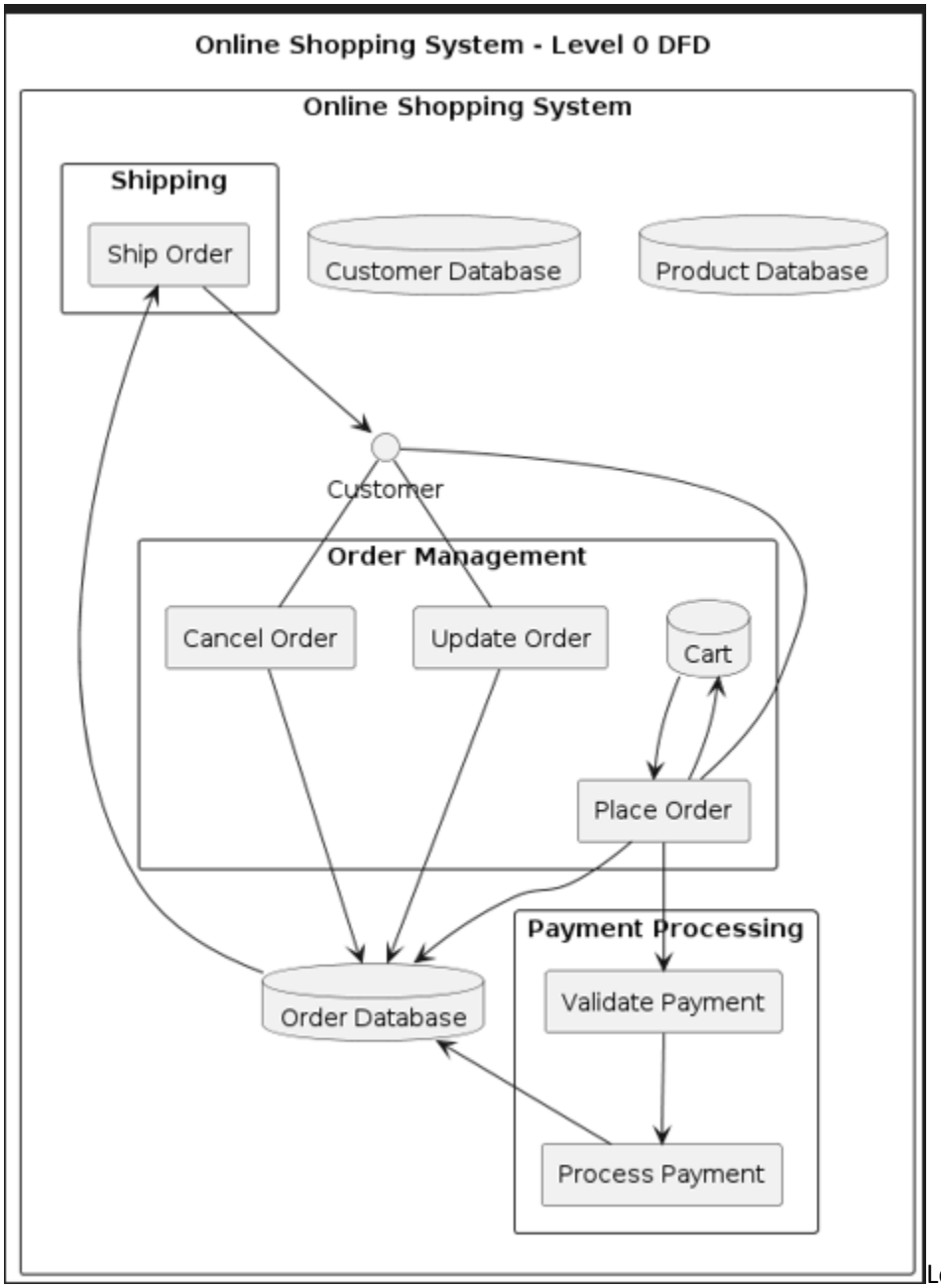
Example for Transaction Mapping

Let's illustrate an example of transaction mapping using a simple online shopping system as described in the context of designing data flow diagrams (DFDs) for software engineering.

Step 1: Create a Level 0 DFD.

At the highest level, the Level 0 DFD illustrates key actions of the online shopping system:

1. **Customer Management**
2. **Product Management**
3. **Order Management**
4. **Payment Processing**
5. **Shipping**



Level 0 DFD

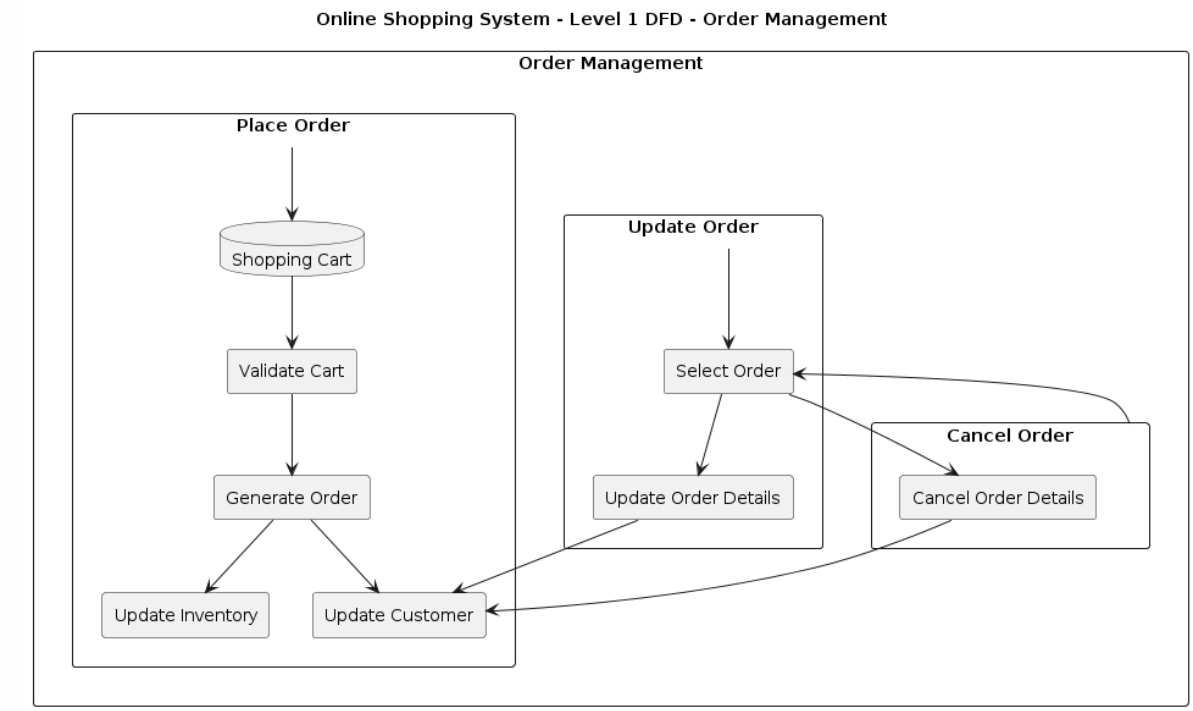
These processes represent the major functionalities of the system.

Step 2: Create Lower-Level DFDs.

For each of the processes identified in the Level 0 DFD, create lower-level DFDs to explain the involved processes in detail. For example:

Order Management Process

- **Sub-process:** Place Order, Update Order, Cancel Order
- **Data Stores:** Order Database
- **Data Flows:** Order Information (from Place Order to Order Database), Updated Order Status (from Update Order to Order Databases



Level 1 DFD

Step 3: Add Data Stores.

Include data stores in the DFD to represent where data is stored within the system. For Example:

- **Customer Database:** Stores customer information like name, address, and contact details.
- **Product Database:** Stores product details such as name, description, price, and availability.
- **Order Database:** Stores information related to customer orders, including order ID, items purchased, quantity, total price, and shipping details.

Step 4: Add Data Flows.

Show data flows between processes using arrows to indicate the movement of data within the system. For example:

- **Data Flow:** 'Order Information' flows from the 'Place Order' process to the Order Database, capturing details of the customer's order.
- **Data Flow:** 'Updated Order Status' flows from the 'Update Order' process to the Order Database, reflecting changes in the order status.

Transaction Mapping

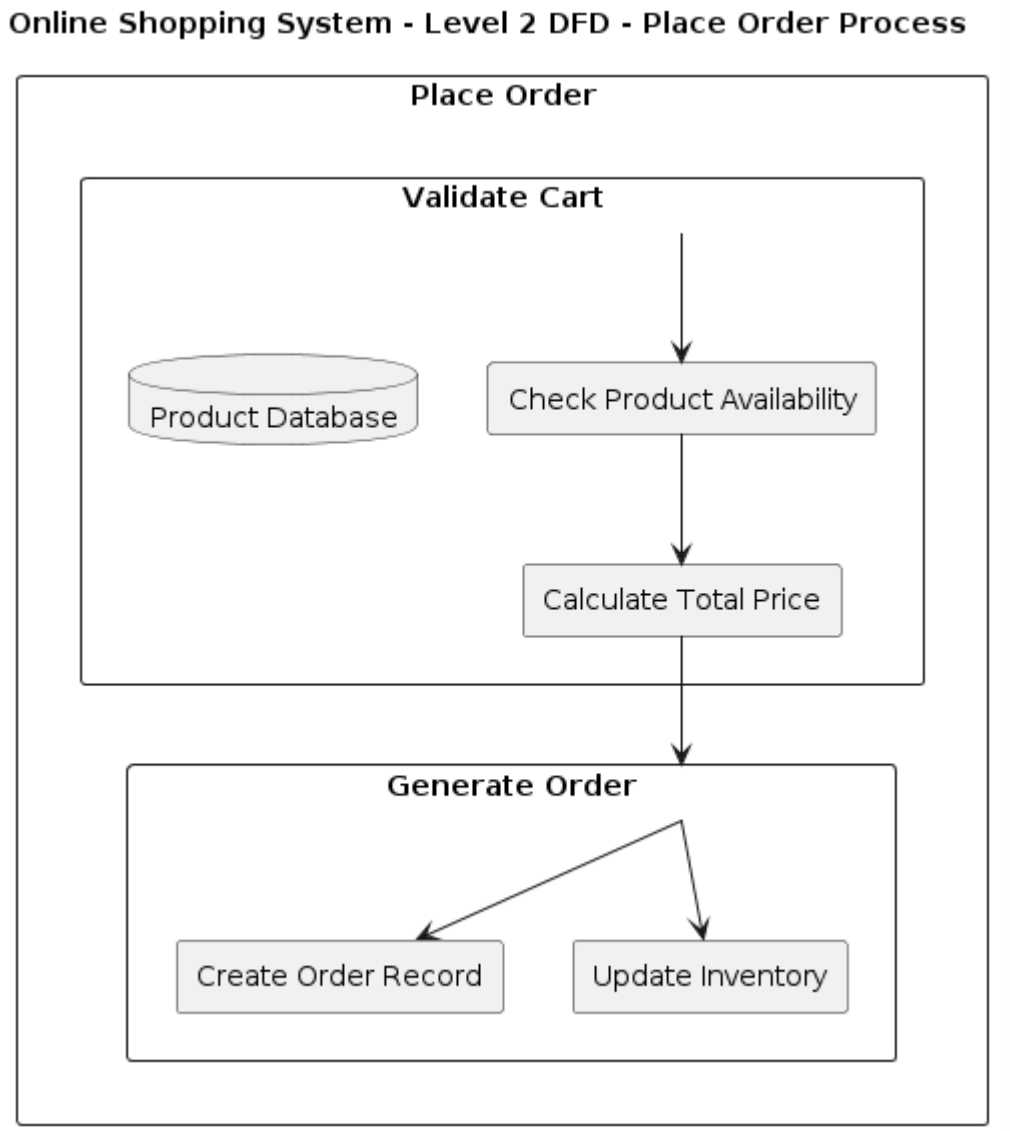
In transaction mapping, each data item or transaction (e.g., placing an order, or updating order status) triggers information flows and functions within the system. For instance:

1. Placing an Order Transaction

- Triggers data flow of '**Order Information**' from the '**Place Order**' Sub-process to the Order Database.
- Initiates processes related to inventory management and order fulfillment.

2. Updating Order Status Transaction

- Triggers data flow of '**Updated Order Status**' from the '**Update Order**' Sub-process to the Order Database.
- May initiate processes related to notifying customers, updating inventory, or generating shipping labels.



Level 2 DFD

Refactoring - Introduction and Its Techniques

Refactoring or **Code Refactoring** is defined as systematic process of improving existing computer code, without adding new functionality or changing external behaviour of the code. It is intended to change the implementation, definition, structure of code without changing functionality of software. It improves extensibility, maintainability, and readability of software without changing what it actually does. **Why should we refactor our code when it works fine?** The goal of refactoring is not to add new functionality or remove an existing one. The main goal of refactoring is to make code easier to maintain in future and to fight technical debt. We do refactor because we understand that getting design right in first time is hard and also you get the following benefits from refactoring:

- Code size is often reduced
- Confusing code is restructured into simpler code

Both of the above benefits greatly improve maintainability which is required because requirements always keep changing. **When do we refactor?**

- Before you add new features, make sure your design and current code is "good" this will help the new code be easier to write.
- When you need to fix a bug
- When you do a peer review
- During a code review

How to identify code to refactor? Martin Fowler proposed using "code smells" to identify when and where to refactor. Code smells are bad things done in code, just like bad patterns in the code. Refactoring and Code smells are a few techniques that help us identify problems in design and implementation. It also helps us in applying known solutions to these problems. **Refactoring Techniques :** There are more than 70 refactoring techniques that exist. But we will discuss only a few, more common ones.

1. **Extract Method -** When we have a code that can be grouped together. **Example:**

```
def student():
   getgrades()
   # details
   name = input()
   class = input()
```

1. This could be refactored as:

```
def student():
   getgrades()
   getdetails()

def getdetails():
   name = input()
   class = input()
```

1. **Replace Temp with Query -** When we are using a temporary variable to hold the result of an expression. **Example :**

```
SI = P * R * T / 100
if(SI > 100):
   return SI
else:
   return SI * 1.5
```

1. This could be refactored as:

```
def calculate_SI(P, R, T):
   return P * R * T / 100
```

```
SI = calculate_SI(P, R, T)

if SI > 100:
   return SI
else:
   return SI * 1.5
```

1. **Encapsulate Field -** It involves providing methods that is used to read/write data rather than accessing it directly. **Example :**

```
class A:
   variable
```

This could be refactored as:

```
class A:
    self.variable
    getvariable()
    setvariable()
class A:
    self.variable
    getvariable()
    setvariable()
```

1. **Inline Method -** When we have a method body which is more obvious than the method itself. **Example :**

```
class PizzaDelivery:
   def getgrades(self):
      return 'A' if self.moretheneight() else B
   def ismorethaneight(self):
      return self.number > 8
```

1. This could be refactored as:

```
class PizzaDelivery:
   def getgrades(self):
      return self.number > 8 ? A : B
```

1. **Move Method -** When a function class is used by other class more than the class in which it exists.

```
Class A:
   #...
   abc()

Class B:
   #...
```

1. This could be refactored as:

```
Class A:
   #...

Class B:
   #...
   abc()
```

1. **Replace Conditional with Polymorphism -** When we have a conditional that performs various actions depending on object type or properties. **Example :**

```
=class Bird:
   # ...
   def getSpeed(self):
      if self.type == EUROPEAN:
         return self.getBaseSpeed()
      elif self.type == AFRICAN:
         return self.getBaseSpeed() - self.getLoadFactor() * self.numberOfCoconuts
      elif self.type == NORWEGIAN_BLUE:
         return 0 if self.isNailed else self.getBaseSpeed(self.voltage)
      else:
         raise Exception("Should be unreachable")
```

1. This could be refactored as

```
class Bird:
   # ...
   def getSpeed(self):
      pass

class European(Bird):
   def getSpeed(self):
      return self.getBaseSpeed()


class African(Bird):
   def getSpeed(self):
      return self.getBaseSpeed() - self.getLoadFactor() * self.numberOfCoconuts


class NorwegianBlue(Bird):
   def getSpeed(self):
      return 0 if self.isNailed else self.getBaseSpeed(self.voltage)

# Somewhere in client code
speed = bird.getSpeed()
```

Note :

- Refactoring improves the design of software.
- Refactoring makes software easier to understand.

- Refactoring helps us finding bugs in the program.
- Refactoring helps us program faster.

Object-Oriented Design (OOD) - System Design

Object-oriented design (OOD) is a programming technique that solves software problems by building a system of interrelated objects. It makes use of the concepts of classes and objects, encapsulation, inheritance, and polymorphism to model real-world entities and their interactions. A system architecture that is modular, adaptable, and simple to understand and maintain is produced using OOD.

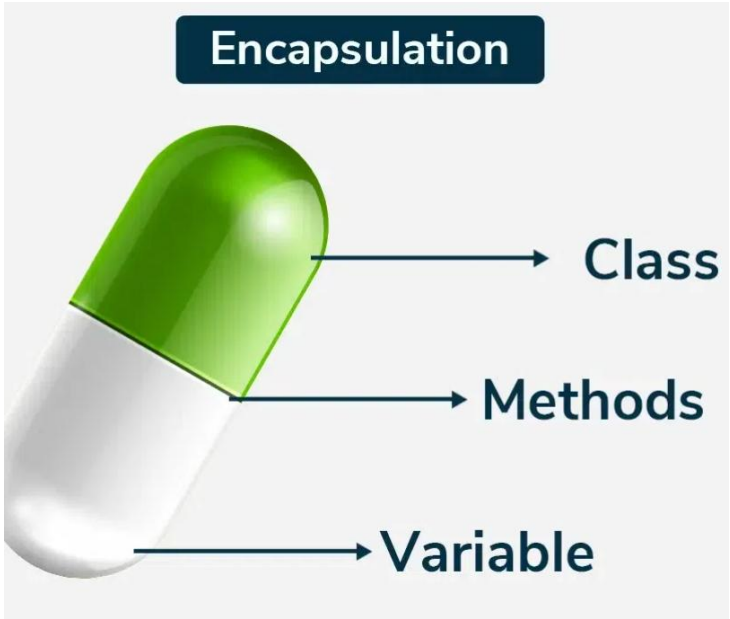
Object-Oriented Design Concepts

A number of fundamental ideas are included in object-oriented design (OOD), which makes it easier to create software that is reliable, scalable, and maintainable. These are the main ideas, supported by examples and explanations.

1. Encapsulation

Encapsulation is the bundling of data (attributes) and methods (functions) that operate on the data into a single unit called a class. It restricts direct access to some of the object's components, which is a means of preventing accidental interference and misuse of the data.

Encapsulation



Example:

Consider a Car class with private attributes speed and fuelLevel and public methods accelerate(), brake(), and refuel(). The internal state of speed and fuelLevel can only be modified through these methods, ensuring controlled access.

```
#include <iostream>


using namespace std;
```

```
class Car {
private:
    int speed;
    double fuelLevel;

public:
    Car() : speed(0), fuelLevel(100.0) {}

    void accelerate(int amount) {
        speed += amount;
        fuelLevel -= amount * 0.1;
    }

    void brake() {
        speed = 0;
    }

    void refuel(double amount) {
        fuelLevel += amount;
    }

    void display() {
        cout << "Speed: " << speed << ", Fuel Level: " << fuelLevel << endl;
    }
};

int main() {
    Car myCar;
    myCar.accelerate(10);
    myCar.brake();
```

```
    myCar.refuel(20);
    myCar.display();
    return 0;
}
```

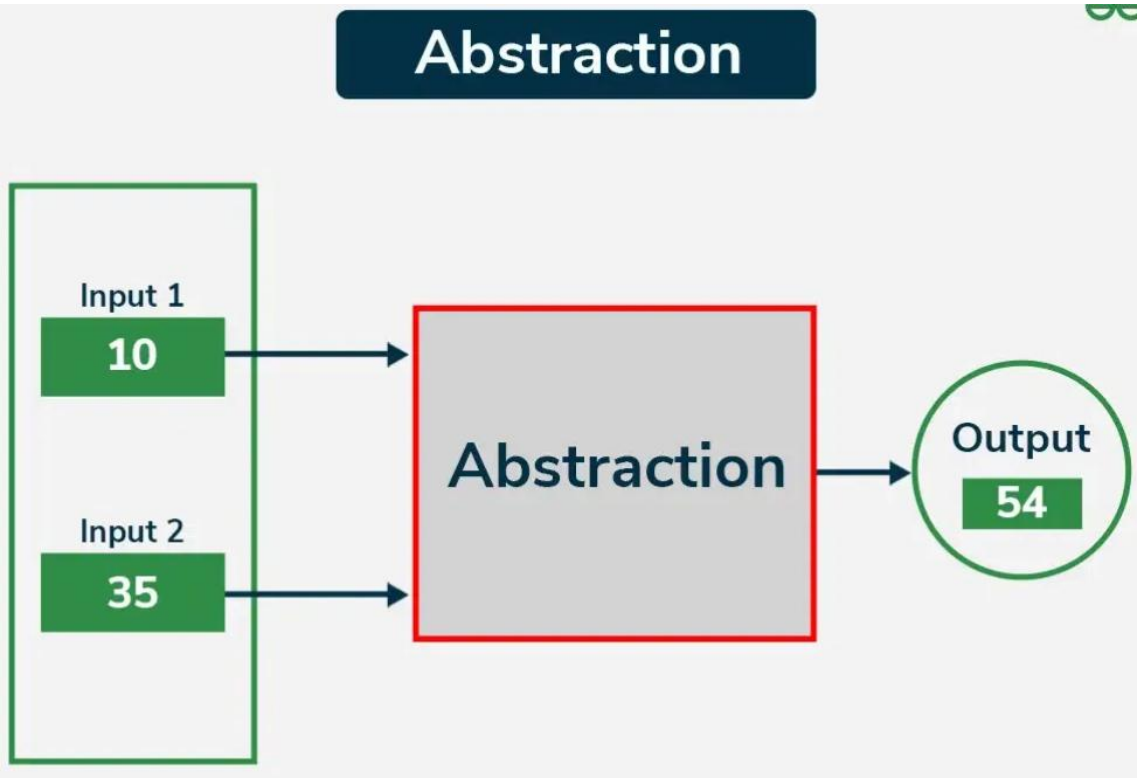
Output

Speed: 0, Fuel Level: 119

2. Abstraction

Abstraction involves hiding the complex implementation details and showing only the essential features of the object. This helps in managing complexity by allowing the designer to focus on the interactions at a higher level.

Abstraction



Example:

An Animal class can represent general properties and behaviors common to all animals without detailing the specific implementation for each type of animal.

```
#include <iostream>
#include <string>

using namespace std;
```

```
class Animal {
public:
    virtual string makeSound() const = 0; // Pure virtual function
};

class Dog : public Animal {
public:
    string makeSound() const override {
        return "Bark";
    }
};

class Cat : public Animal {
public:
    string makeSound() const override {
        return "Meow";
    }
};

int main() {
    Animal* dog = new Dog();
    Animal* cat = new Cat();

    cout << dog->makeSound() << endl;
    cout << cat->makeSound() << endl;

    delete dog;
    delete cat;
    return 0;
}
```

Output

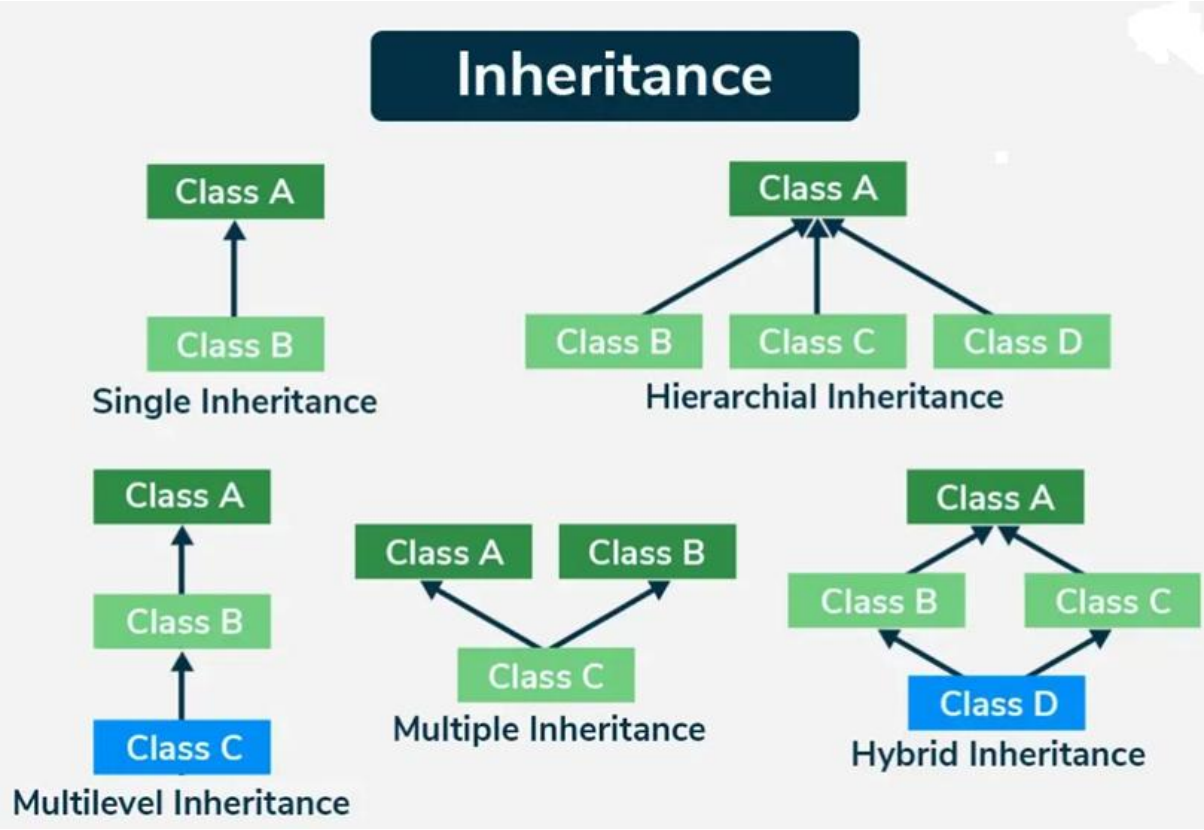
Bark

Meow

3. Inheritance

Inheritance is a mechanism where a new class inherits properties and behaviors (methods) from an existing class. This promotes code reuse and establishes a natural hierarchy between classes.

Inheritance



Example:

A Vehicle class can be a parent class with common attributes like make and model, and child classes like Car and Bike can inherit these attributes and have additional specific properties

```
#include <iostream>

#include <string>


using namespace std;


class Vehicle {

protected:
```

```
    string make;
    string model;

public:
    Vehicle(std::string make, std::string model) : make(make), model(model) {}
};

class Car : public Vehicle {
private:
    int num_doors;

public:
    Car(string make, string model, int num_doors) : Vehicle(make, model), num_doors(num_doors) {}

    void display() {
        cout << "Make: " << make << ", Model: " << model << ", Doors: " << num_doors << endl;
    }
};

class Bike : public Vehicle {
private:
    string type_bike;

public:
    Bike(string make, string model, string type_bike) : Vehicle(make, model), type_bike(type_bike) {}

    void display() {
        cout << "Make: " << make << ", Model: " << model << ", Type: " << type_bike << endl;
    }
};
```

```
int main() {
    Car myCar("Toyota", "Corolla", 4);
    Bike myBike("Yamaha", "MT-07", "Sport");

    myCar.display();
    myBike.display();
    return 0;
}
```

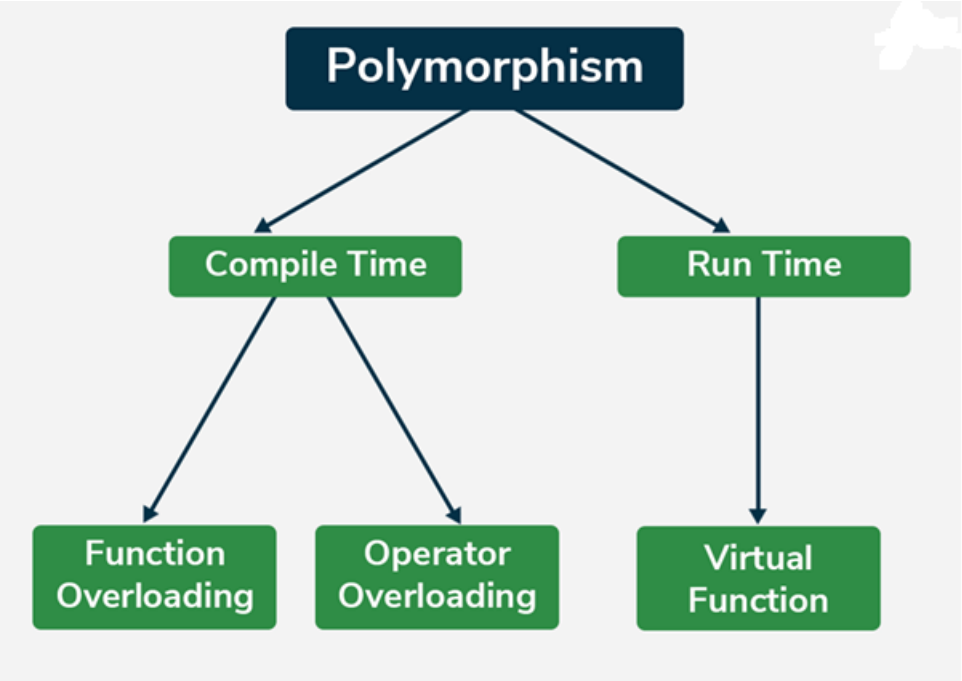
Output

Make: Toyota, Model: Corolla, Doors: 4

Make: Yamaha, Model: MT-07, Type: Sport

4. Polymorphism

Polymorphism allows objects of different classes to be treated as objects of a common super class. It enables a single interface to represent different underlying data types and allows methods to use objects of various types.



Polymorphism

Example:

Both Dog and Cat classes inherit from Animal and implement the make_sound method. A function can take an Animal object and call make_sound, regardless of whether it's a Dog or Cat.

```
#include <iostream>
#include <string>

using namespace std;

class Animal {
public:
    virtual string makeSound() const = 0; // Pure virtual function
};

class Dog : public Animal {
public:
    string makeSound() const override {
        return "Bark";
    }
};

class Cat : public Animal {
public:
    string makeSound() const override {
        return "Meow";
    }
};

void animalSound(const Animal& animal) {
    cout << animal.makeSound() << endl;
}

int main() {
    Dog dog;
    Cat cat;
```

```
    animalSound(dog); // Outputs: Bark
    animalSound(cat); // Outputs: Meow
    return 0;
}
```

Output

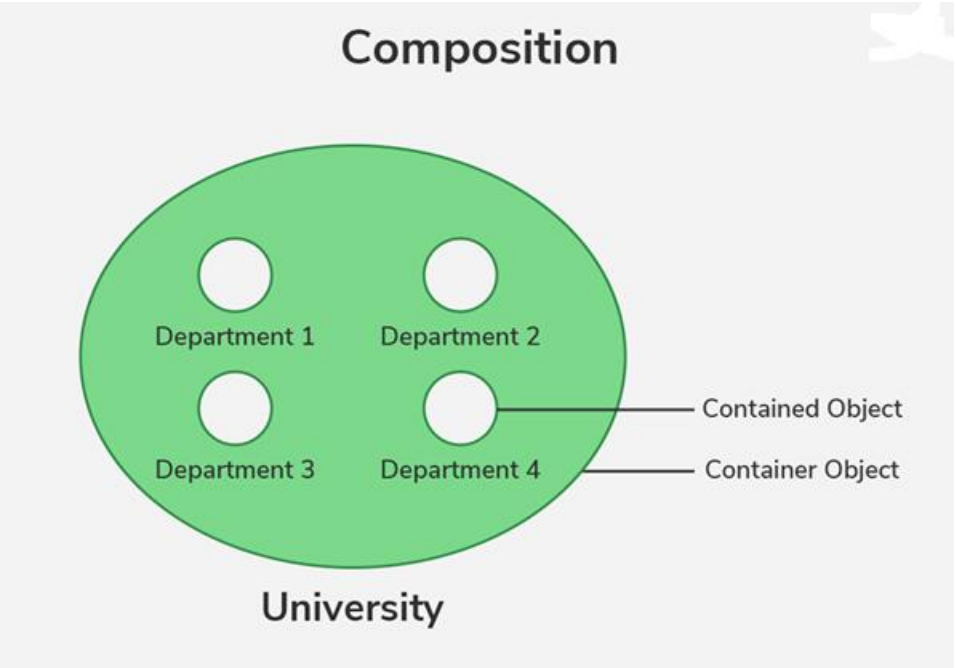
Bark

Meow

5. Composition

Composition is a design principle where a class is composed of one or more objects of other classes, rather than inheriting from them. This promotes flexibility and reusability.

Composition



Example:

A Library class can be composed of Book objects. Instead of inheriting from Book, the Library class contains multiple Book instances.

```
#include <iostream>
#include <vector>
#include <string>
```

```
using namespace std;

class Book {
private:
    string title;
    string author;

public:
    Book(string title, string author) : title(title), author(author) {}

    string getTitle() const {
        return title;
    }

    std::string getAuthor() const {
        return author;
    }
};

class Library {
private:
    vector<Book> books;

public:
    void addBook(const Book& book) {
        books.push_back(book);
    }

    void listBooks() const {
        for (const auto& book : books) {
            cout << book.getTitle() << " by " << book.getAuthor() << endl;
```

```
        }
    }
};

int main() {
    Book book1("1984", "George Orwell");
    Book book2("To Kill a Mockingbird", "Harper Lee");

    Library library;
    library.addBook(book1);
    library.addBook(book2);
    library.listBooks();
    return 0;
}
```

Output

1984 by George Orwell

To Kill a Mockingbird by Harper Lee

By understanding and applying these core concepts, developers can create well-structured, maintainable, and efficient software systems.

Design Patterns in Object-Oriented Design (OOD)

Design patterns in Object-Oriented Design (OOD) are proven solutions to common problems that arise in software design. These patterns provide templates that help to structure code in an efficient and maintainable way.

Here are some of the most commonly used design patterns in OOD:

1. Creational Patterns: Creational patterns deal with object creation mechanisms, trying to create objects in a manner suitable to the situation. Some key creational patterns include:

- **Singleton**: Ensures a class has only one instance and provides a global point of access to it.
- **Factory Method**: Defines an interface for creating an object, but lets subclasses alter the type of objects that will be created.

2. Structural Patterns: Structural patterns deal with object composition or structure, ensuring that if one part changes, the entire structure does not need to do so. Some key structural patterns include:

- **Adapter**: Converts the interface of a class into another interface the clients expect. It lets classes work together that couldn't otherwise because of incompatible interfaces.

- **Composite**: Composes objects into tree structures to represent part-whole hierarchies. It lets clients treat individual objects and compositions of objects uniformly.

3. Behavioral Patterns: Behavioral patterns deal with communication between objects, making it easier and more flexible. Some key behavioral patterns include:

- **Observer**: Defines a one-to-many dependency between objects so that when one object changes state, all its dependents are notified and updated automatically.
- **Strategy**: Defines a family of algorithms, encapsulates each one, and makes them interchangeable. It lets the algorithm vary independently from clients that use it.

UML Diagrams for Visualizing OOD

Diagrams created using the Unified Modeling Language (UML) are useful tools for understanding and clarifying object-oriented system structure. They serve as a blueprint, showing the interactions between many components of a system, which makes difficult concepts simpler to understand and communicate.

Here are some common UML diagrams and how they contribute to object-oriented design:

- **Class Diagrams**: These diagrams show the structure of a system by highlighting the classes, their attributes, and relationships. This is useful in visualizing how data and behaviors are organized, which is the foundation of object-oriented design.
- **Sequence Diagrams**: Sequence diagrams illustrate how objects in a system interact over time. They show the flow of messages and actions, helping designers see the order in which things happen. This makes it easier to understand the dynamic behavior of a system.
- **State Diagrams**: State diagrams represent the various states of an object and the transitions between them. They help designers understand how objects respond to different events, which is key for designing systems that change over time.

By using these diagrams, developers can make sure everyone has a clear picture of the system's structure and behavior, making it easier to collaborate and avoid misunderstandings.

Common Challenges and Anti-Patterns in Object-Oriented Design (OOD)

While it offers strong software development concepts, object-oriented design (OOD) is not without its challenges and drawbacks. For systems to be productive and maintained, it is essential to recognize these difficulties and stay clear of typical anti-patterns.

These are a few common OOD challenges and anti-patterns:

Common Challenges in Object-Oriented Design (OOD)

1. Designing for potential future needs that may never arise, which adds needless complexity.

- **Impact**: Leads to code that is harder to understand, maintain, and extend.
- **Mitigation**: Focus on current requirements and implement extensibility only when there's a clear need.

2. Not foreseeing future demands and modifications, which leads to an inflexible system.

- **Impact**: Makes the system difficult to extend or modify.

- **Mitigation**: Apply principles like SOLID and design patterns that facilitate flexibility and scalability.

3. Ensuring that encapsulation does not excessively degrade performance.

- **Impact**: Encapsulation can lead to additional layers of abstraction that may impact performance.
- **Mitigation**: Use encapsulation judiciously and optimize critical performance paths as needed.

4. Determining the right level of abstraction to balance simplicity and functionality.

- **Impact**: Too much abstraction can obscure functionality; too little can lead to code duplication.
- **Mitigation**: Aim for clear and concise abstractions that accurately represent the problem domain.

Common Anti-Patterns in Object-Oriented Design (OOD)

- **God Object/Anti-Pattern**: A single class takes on too many responsibilities, violating the Single Responsibility Principle.
- **Spaghetti Code**: Code with a complex and tangled control structure, making it difficult to follow and maintain.
- **Lava Flow**: Dead code, outdated design elements, and obsolete components that remain in the codebase.
- **Object Orgy**: Excessive sharing of data and methods between classes, leading to tight coupling and lack of encapsulation.